

DB (Database)

 THAPA TECHNICAL

Database

A database is just a collection of data stored in an organized way.

-  Student records
-  User accounts
-  Orders in an e-commerce app

 Think of it as: “The actual stored data.”

DBMS (Database Management System)

 THAPA TECHNICAL

DBMS

A DBMS is software used to create, manage, and access databases.

What it helps you do

-  Store data
-  Update data
-  Delete data
-  Query data

Examples

- MySQL
- PostgreSQL
- MongoDB
- SQLite

RDBMS (Relational Database Management System)

 THAPA TECHNICAL

An RDBMS is a special type of DBMS

What is RDBMS?

- Stores data in tables
- Organized into rows and columns
- Uses relationships between tables

Key Features ✨

- Uses SQL
- Supports relations between tables
- Has primary keys & foreign keys
- Follows ACID properties

Example:

Users Table

id	name
1	Vinod

Orders Table

id	user_id
1	1

  Here, **user_id** connects **Orders** → **Users**. That relationship between tables is what makes it an **RDBMS**.

SQL (Structured Query Language)

 THAPA TECHNICAL

SQL: The Language of Databases

SQL is a language used to communicate with databases.

 Insert data

 Read data

 Update data

 Delete data

 SQL is **not a database software**. It is a **language**.

What is PostgreSQL?

PostgreSQL is an RDBMS.

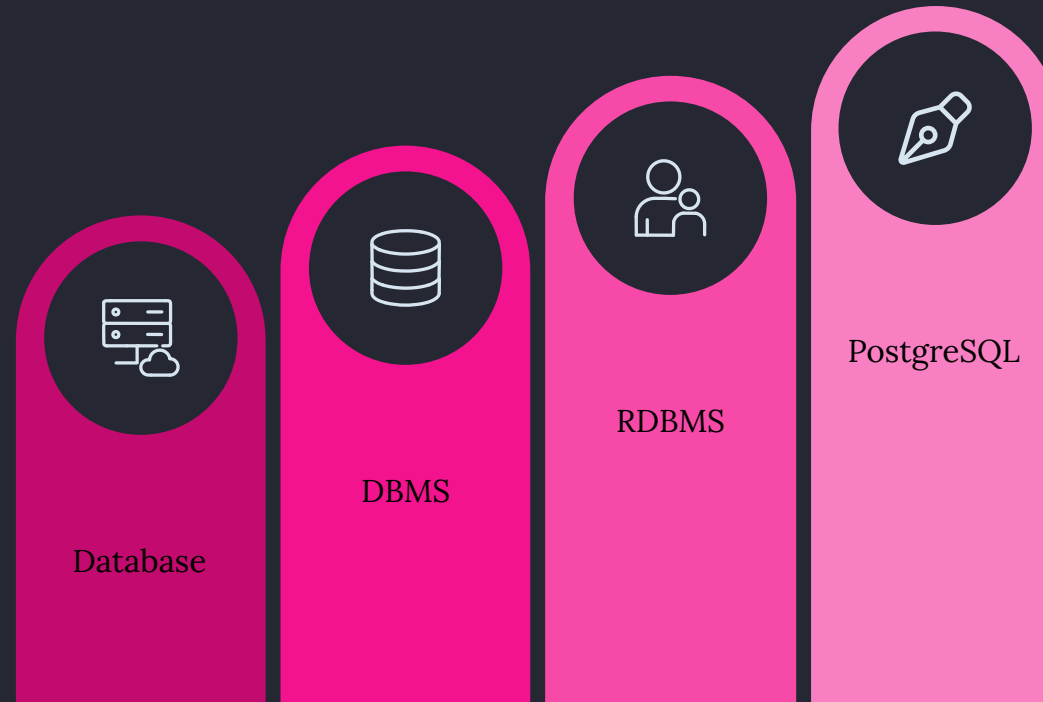
It is **advanced**, **open-source**, and uses **SQL** to manage **relational data**.

 ✨ Think of PostgreSQL as a powerful database system for storing and organizing structured information.

Where does PostgreSQL fit?

 THAPA TECHNICAL

PostgreSQL is a specific kind of database software



-  **Database** = the data itself
-  **DBMS** = software to manage databases
-  **RDBMS** = DBMS with relational tables
-  **PostgreSQL** = a specific RDBMS

Excel vs. PostgreSQL

Both store data in rows and columns, but PostgreSQL adds relationships between tables.

-  **Excel:** acts like a simple database
-  **Rows:** records
-  **Columns:** fields
-  **PostgreSQL:** relational database
-  **Key difference:** supports relationships between tables



Excel Sheet as a Basic Database

This sample sheet shows how spreadsheet data maps to database concepts.

ID	Name	Age	Meaning
1	Vinod	23	One record
2	Sita	21	One record
3	Ravi	24	One record

Key idea

- Each **row** is one **record**.
- Each **column** is one **field**.
- The full sheet can be treated as a basic **database table**.



Complete PostgreSQL 18 Installation on Windows 11 & CLI Setup

Step-by-step setup for installing PostgreSQL 18 on Windows 11 and preparing the command-line tools for use.

 THAPA TECHNICAL



Made with GAMMA



Today on our agenda



Critical configurations

Watch out for important settings during installation.



EDB installer walkthrough

Step-by-step setup on Windows 11.



Environment Variables & PATH

Understand the 2-layer explanation clearly.



Verification checks

Run CLI and GUI validation checks.

⚡ Crucial Checklist Before You Click 'Next'



🔑 The Superuser Password

PostgreSQL creates a master account named `postgres`.

Write this down. If you lose it, you cannot access your local databases.

For development, use something memorable like `admin123` or `postgres`.



🧭 The Port Number

By default, PostgreSQL communicates on port `5432`.

Unless another version of Postgres is already running, **do not change this number.**



🌐 The Locale

Keep this set to `[Default locale]`.

This ensures your system handles text characters and alphabet sorting exactly the way your Windows OS expects.



⚡ Terminal Tools Live in C:\Program Files\PostgreSQL\18\bin

Adding this folder to your Windows Environment variables tells Windows where to find `psql` from anywhere on your computer.

ⓘ Think of PATH like a search list

When you type a command in the terminal, Windows checks the folders in PATH until it finds the program.

1. Install PostgreSQL

The command tools are placed inside the `bin` folder.

2. Add the folder to PATH

Windows now knows to look in `C:\Program Files\PostgreSQL\18\bin`.

3. Run tools from anywhere

Type `psql` from any directory and Windows finds it automatically.

Quick Understanding Check

Use these two questions to confirm the key idea: **PATH points to the tool location, not the database name.**

1. Why won't Windows run `psql` right away?

After installation, Windows cannot find `psql` until the PostgreSQL `bin` folder is added to the environment variables.

2. Do we need to update PATH if the database name changes?

No. If we rename the database to `college_management`, the Windows system PATH does **not** need to change.



Hint: PATH tells Windows where to find `psql`. It does **not** depend on the database name.



⚡ Verification Check 1: The CLI Terminal Test

Use this command to connect to PostgreSQL from the terminal:

❗ **Syntax:** `psql -U username`

`psql`

Launches the PostgreSQL Command Line Interface tool in Windows.

`-U flag`

Stands for **User** and tells PostgreSQL which account is trying to log in.

`username`

The placeholder for the database user account.
The default master account is `postgres`.

Examples

- `psql -U postgres` — logs in using the default PostgreSQL superuser.
- `psql -U admin` — logs in with a different database username.

✅ **Quick tip:** Replace `username` with the actual PostgreSQL user you created or were given.



DATABASE

Core commands for creating, connecting to, listing, and removing databases in PostgreSQL.

CREATE DATABASE

Creates a new database named `myapp`.

```
CREATE DATABASE myapp;
```

CREATE IF NOT EXISTS

Safely creates the database only when it does not already exist.

```
CREATE DATABASE IF NOT  
EXISTS myapp; -- PostgreSQL  
17+
```

CONNECT

Switch to the `myapp` database in `psql`.

```
\c myapp -- connect (psql)
```

LIST DATABASES

Shows all databases currently available on the server.

```
\l -- list all databases
```

DROP DATABASE

Deletes the `myapp` database permanently.

```
DROP DATABASE myapp;  
DROP DATABASE IF EXISTS  
myapp;
```



Key Database Concepts

Core building blocks that help organize data clearly and efficiently in PostgreSQL.



Table

A structured container inside your database holding data about one specific entity, like Students, Courses, or Departments.



Column / Field

The vertical categories in a table that define what kind of data is stored, such as `name`, `email`, or `age`.



Row / Record

The horizontal entry in a table. Each row represents one complete, distinct item, like one specific student's full profile.



Primary Key

A unique identifier for each row in a table. It makes every record easy to find and prevents duplicates.

Remember: Tables store entities, columns define properties, rows hold records, and primary keys keep everything uniquely identified.





Basic Syntax

Creating a table

Define a new table and its columns.

```
CREATE TABLE table_name (  
    column1_name data_type,  
    column2_name data_type  
);
```

Deleting a table

Remove an existing table from the database.

```
DROP TABLE table_name;
```

Listing tables

Show all tables in the current database session.

```
\dt
```



THAPA TECHNICAL



Best Practices, Tips & Tricks

Day 3 focus: avoid common PostgreSQL mistakes and build clean, consistent table definitions.

⚠ The Trailing Comma Trap

A very common beginner mistake is leaving a comma after the *last* column in your `CREATE TABLE` statement. PostgreSQL will throw a syntax error. The last item never gets a comma!

🏷 Singular vs. Plural

Pick a naming convention for your tables and stick to it. If you name one table `users` (plural), don't name the next one `course` (singular). In this series, we will use plural table names (`users`, `courses`).

✎ Don't Over-allocate VARCHAR

While `VARCHAR(255)` is a common default habit from other databases, if you know a column (like a phone number or a specific ID code) will never exceed 15 characters, use `VARCHAR(15)`. It acts as a helpful built-in constraint against bad data.

+ INSERT (Create)

pgAdmin4 overview: use INSERT to add new rows, speed up writes with batch insert, and handle outcomes with PostgreSQL's RETURNING and ON CONFLICT clauses.

INSERT

The SQL command used to add new records into an existing table.

Batch Insert

Insert multiple records in a single statement to reduce network overhead and improve execution speed.

RETURNING Clause

Instantly returns specified columns from the newly created row without needing a separate `SELECT` query.

ON CONFLICT Clause

Handles unique constraint conflicts by telling PostgreSQL whether to skip the row or update existing data, helping prevent application errors.



Explicit Column Insertion

```
INSERT INTO table_name (col1, col2, col3) VALUES (val1, val2, val3);
```

INSERT INTO

Points to the destination table where the new row will be added.

Column List

Defines exactly which fields you are filling, in the exact order you want.

VALUES

Supplies the data for each column, matched position-by-position.

Example 1

```
INSERT INTO students (id, name, grade) VALUES (101, 'Ava', 'A');
```

Example 2

```
INSERT INTO products (product_id, product_name, price) VALUES  
(12, 'Keyboard', 49.99);
```

Use explicit column insertion when you want full control over which fields receive data and in what order.



THAPA TECHNICAL



Multi-row Batch Insertion with RETURNING

Use one INSERT statement to create multiple rows at once, then immediately get back generated values like IDs and timestamps.

```
INSERT INTO table_name (col1, col2)
VALUES
  (valA1, valA2),
  (valB1, valB2)
RETURNING id, created_at;
```

INSERT INTO

Targets the destination table where new rows will be created.

VALUES (...), (...)

Each parenthesized group is a separate row. Commas separate the rows in the batch.

RETURNING id, created_at

PostgreSQL sends back generated values right away, so you can confirm what was created without another query.

Why it is useful: RETURNING is great for capturing auto-generated primary keys, timestamps, or defaults immediately after insert. It reduces extra round trips and keeps write + read flows in one statement.



Importing Massive Datasets (Excel & CSV)

When launching a new college term, manually typing thousands of rows is impossible. Data is provided via Excel or CSV sheets.

The pgAdmin 4 Import Utility

1. Right-click your users table
2. Select **Import/Export Data**
3. Toggle to **Import**
4. Select your .csv file
5. Enable **Header**
6. Map your Excel/CSV headers to your schema columns

The High-Speed Production Way (psql \copy CLI)

The absolute fastest server-side tool available. It can digest millions of rows in milliseconds.

```
\copy users(name, email, role,  
department_id) FROM  
'/path/to/students.csv' DELIMITER ',' CSV  
HEADER;
```



Practice Questions – Insert

Practice writing `INSERT` queries for single rows, multiple rows, and conflict handling.

Add a New Course

Write a simple query to add a new course named 'Introduction to Psychology' with a fee of 1500.00 into the `courses` table.

Sweet Hint: Because the `id` and `created_at` columns are generated automatically by the database, you only need to specify and insert the `title` and `fee`.

Multiple Departments at Once

Insert two new departments—'Civil Engineering' and 'Mathematics'—in one single command. Make sure the query instantly displays the auto-generated `id` for both of them.

Sweet Hint: Put each department name inside its own set of parentheses separated by a comma. End your query with the special keyword that "returns" the `id`.

No Crashing Allowed

Try to add a new user named 'Super Admin' with the email 'admin@college.com'. Write the query so that if this exact email already exists in the system, the database just ignores it instead of showing an ugly red error.

Sweet Hint: You need to tell the database what to do `ON CONFLICT` with the `(email)` column. Tell it to just do nothing!

👁️ SELECT (Read)

SELECT

SELECT asks the database a question and fetches specific data from your tables. It's the "Read" in CRUD.

* Asterisk (*)

The wildcard character that means "Give me absolutely every column in this table."

🏷️ AS Alias

A temporary nickname you give to a column or table just for the final output, making it easier for humans to read.

🔍 DISTINCT

A keyword that filters out duplicate values, ensuring your output only shows unique, one-of-a-kind results.

🔗 String Concatenation (||)

Gluing two or more pieces of text together directly inside the database before sending it to your frontend.



Syntax & Breakdown

Give Me Everything

```
SELECT * FROM  
table_name;
```

Use `*` to return every column from the table.

Selecting Specific Columns

```
SELECT column1,  
column2 FROM  
table_name;
```

Choose only the fields you actually need for a cleaner result.

Using Aliases

```
SELECT column1  
AS "Custom  
Name" FROM  
table_name;
```

Rename outputs temporarily to make them easier to read.

Eliminating Duplicates

```
SELECT DISTINCT  
column_name  
FROM  
table_name;
```

Remove repeated values so the result shows unique entries only.



Best Practices, Tips & Tricks (Day 5 Focus)

Stop Using `SELECT *` in Production

It is great for quick debugging, but in a real Next.js/Node.js app, always specify your columns.

```
SELECT id, name FROM  
table_name;
```

If a table has a massive JSONB column or huge text blocks, `SELECT *` will slow your application down drastically by fetching data you don't even need.

Single Quotes vs. Double Quotes (Crucial!)

Single quotes (') are for **data/values**, while double quotes (") are for **database identifiers/aliases**.

```
SELECT 'student' AS "Role";  
SELECT first_name AS "Full  
Name" FROM students;
```

Mixing them up is a guaranteed syntax error.

Math Follows BODMAS/PEMDAS

If you do complex math in a `SELECT` statement, use parentheses to control the order of operations.

```
SELECT (fee + 100) * 1.10 FROM  
payments;
```

This ensures your calculation behaves exactly like standard mathematics.

Practice Questions

1

The Contact List

Write a query to retrieve only the `name`, `email`, and `is_active` columns from the `users` table, but alias them as "User Name", "Contact", and "Status".

Sweet Hint: Don't forget to use double quotes around your aliases since they contain spaces!

2

Discover the Departments

We have users assigned to different departments. Write a query to find a clean list of all the unique `department_id` numbers currently in use inside the `users` table.

Sweet Hint: You need to eliminate duplicates. Think of the keyword that means "clearly different".

3

The Welcome Message

Write a query that creates a virtual column called "Welcome Message". It should glue together the string 'Welcome to the college, ' and the user's name.

Sweet Hint: Remember to use single quotes for your plain text string, and the double-pipe `||` symbol to glue it to the column!



UPDATE (Update)



UPDATE

The SQL command used to modify or change data that *already exists* inside your tables. It's the "U" in CRUD.



SET

The keyword that tells the database exactly which columns to change and what their new values should be.



WHERE

The filter condition. It tells PostgreSQL *exactly which row(s)* to update.

Without this, you will overwrite the entire table.



RETURNING

Just like we used in INSERT, this instantly hands back the data you just updated so your frontend application can show the new changes immediately.

⚡ A. Updating a Single Field

Use `UPDATE` when you want to change one specific value in an existing row.

```
UPDATE table_name  
SET column_name = 'new_value'  
WHERE condition;
```

UPDATE

The action that tells PostgreSQL to modify existing data.

- Targets rows already in the table
- Works with `SET` to apply changes

SET

Defines the column and the new value you want to write.

- Can update one or many columns
- Must include the new value

WHERE

Filters the exact row or rows to update.

- Prevents accidental full-table updates
- Always double-check this condition

⚠ **Example:** `UPDATE users SET email = 'new@email.com' WHERE id = 42;`

⊗ **Important:** If you forget `WHERE`, PostgreSQL will update every row in the table.

⚡ B. Updating Multiple Fields

Use **UPDATE** when you need to change more than one column in the same row.

```
UPDATE table_name  
SET column1 = 'new_val1', column2 = 'new_val2'  
WHERE condition;
```

Comma Separator

The `,` separates each column assignment inside the `SET` clause.

Multiple Columns

Each column gets its own new value, all in one update statement.

Example

```
UPDATE users SET  
first_name = 'Ava',  
last_name = 'Stone' WHERE  
id = 7;
```

Result

Updates both `first_name` and `last_name` for the matching row.

 Without `WHERE`, every row in the table will be updated.

C. Updating Based on Calculations (Math)

```
UPDATE table_name  
SET numeric_column = numeric_column * 1.10  
WHERE condition;
```

How it works

Use math expressions in the SET clause to increase, decrease, or recalculate values directly in the table.

Common operators

+ add, - subtract, * multiply, / divide.

Example: raise prices 10%

```
UPDATE products  
SET price = price * 1.10  
WHERE category = 'Books';
```

Example: reduce stock by 5

```
UPDATE inventory  
SET quantity = quantity - 5  
WHERE product_id = 42;
```

Math-based UPDATE statements are great when you need to adjust values consistently across many rows. Always include a WHERE clause unless you want to update every row in the table.



Best Practices, Tips & Tricks (Day 6 Focus)

The "SELECT First" Rule (Crucial Tip)

Before you ever run an `UPDATE` query, write a `SELECT *` query using the exact same `WHERE` clause. This lets you physically *see* the rows you are about to modify.

Once you confirm it's the correct data, change the `SELECT *` to `UPDATE`.

```
SELECT *  
FROM table_name  
WHERE condition;
```

Target the Primary Key

Whenever possible, use the `id` column in your `WHERE` clause (e.g., `WHERE id = 5`). Names and emails can sometimes have duplicates or typos, but the Primary Key is mathematically guaranteed to target only that specific row.

```
UPDATE users  
SET email = 'new@email.com'  
WHERE id = 5;
```

Updating to NULL

If you need to clear out data (like removing a user's metadata), you don't use a blank string `"`. You must use the keyword `NULL` (without quotes!).

```
UPDATE users  
SET metadata = NULL  
WHERE id = 10;
```



Practice Questions - Update

1

1

The Promotion

A student named Rahul Sharma has just been hired as a teaching assistant. Update his role to 'professor'.

Hint: Find his ID using a `SELECT` query first, then use a `WHERE` clause with his specific id so you don't accidentally promote everyone.

2

2

The Typo Fix

You accidentally made a typo in a course. Update both the title to 'Advanced AI & Machine Learning' and the fee to 7000.00 for the course with id = 5.

Hint: Use a comma to separate the columns in `SET`, and don't forget `RETURNING *` to instantly display the updated row.

3

3

The Scholarship Discount

The college is offering a flat 500 discount on all premium courses. Write a query that subtracts 500 from the fee of any course that currently costs more than 5000.00.

Hint: You can do math directly in the `SET` clause: `fee = fee - 500`.



DELETE (Delete)



DELETE

The SQL command used to permanently remove existing rows from a table. It deletes row by row, which is safe but slower for massive datasets.



Soft Delete

A professional development practice where you don't actually delete the row. Instead, you `UPDATE` a status column like `is_active = false` to hide the data from the frontend while keeping it in the database for historical records.



Hard Delete

Actually running the `DELETE` command to permanently wipe the data from the hard drive.



TRUNCATE

The "Nuclear Option." It is a special command that instantly empties an entire table without scanning row by row. It is lightning-fast but cannot be undone or filtered with a `WHERE` clause.





The Standard DELETE (Row by Row)

```
DELETE FROM table_name  
WHERE condition;
```

DELETE FROM

The command that starts the removal process.

table_name

The specific table you want to target.

WHERE

CRITICAL! Filters exactly which rows are deleted.



Example

```
DELETE FROM users WHERE id = 7;
```



Warning

Without a `WHERE` clause, every row in the table will be deleted.



Best Practice

Test your filter with `SELECT` first before running `DELETE`.

⚡ The TRUNCATE Command (Wipe the Table)

```
TRUNCATE TABLE table_name;
```

What it does

TRUNCATE immediately drops all data pages associated with the table.

Why it's fast

It removes the table's data in one quick operation instead of deleting rows one by one.

Danger zone

Extremely fast, but dangerous.
Use it only when you truly want to wipe everything.

Compared with DELETE

DELETE removes rows selectively with WHERE; TRUNCATE clears the whole table.

⚠ **Warning:** TRUNCATE does not target individual rows. If you need to remove only some records, use DELETE with a WHERE clause instead.

Fast

Best for clearing an entire table instantly.



Risky

No row-by-row selection, so mistakes are costly.



Different from DELETE

DELETE is surgical; TRUNCATE is a full wipe.



Best Practices, Tips & Tricks (Day 7 Focus)

✓ Always SELECT Before You DELETE

Just like with UPDATE, strongly advise your viewers to write `SELECT * FROM users WHERE...` first. Once they verify only the correct rows show up, replace `SELECT *` with `DELETE`.

```
SELECT * FROM users WHERE ...;  
DELETE FROM users WHERE ...;
```

🔑 Rely on Primary Keys

If you need to delete a user named "Rahul Sharma," never use `WHERE name = 'Rahul Sharma'`. There might be three Rahuls! Always use `WHERE id = 10` to ensure precision.

```
DELETE FROM users  
WHERE id = 10;
```



Practice Questions - Delete

QUESTION 1

The Specific Expulsion

1

Delete a specific user from the `users` table using their unique `id` (for example, ID 50). Make sure you return the deleted row's data.

 *Hint:* Use a `WHERE` clause targeting the `id`, and finish with `RETURNING *`.

QUESTION 2

Course Cleanup

2

Delete all courses in the `courses` table that have a fee of strictly less than 2500.00.

 *Hint:* Use the standard less-than symbol `<` in your `WHERE` clause.

QUESTION 3

The Total Wipe

3

You filled a temporary table with junk data. Write the command to completely empty the table without deleting the table structure itself.

 *Hint:* Don't use `DELETE` or `DROP`. Think of the fast reset command that starts with `TRUNCATE`.



Precision Filtering with WHERE



WHERE Clause

The gatekeeper clause. It filters your query so it only affects or retrieves rows that meet a specific condition.



Operators

`=`, `!=`, `<`, `>`, `<=`, `>=` are the comparison symbols used to evaluate data.



Comparison Meanings

- `=`: Exactly equals.
- `!=` (or `<>`): Does NOT equal.
- `<` / `>`: Strictly less than / strictly greater than.
- `<=` / `>=`: Less than or equal to / greater than or equal to.



Logical Operators

`AND` means both conditions must be true. `OR` means either condition can be true. `NOT` flips the logic of a condition.



WHERE Clause Patterns

Use **WHERE** to filter rows with precision, combine conditions logically, and exclude unwanted results.

Single Condition Filtering

```
SELECT column_name
FROM table_name
WHERE column = 'value';
```

Breakdown: one column, one comparison, one matching value.

- **column_name** — the field you want to filter on
- **table_name** — the table you are querying
- **WHERE** — starts the filtering rule
- **column = 'value'** — keeps only matching rows

Example: find rows where the status is active.

Combining with AND / OR

```
SELECT column_name
FROM table_name
WHERE condition1 AND condition2;
```

Breakdown: combine two conditions to narrow or broaden results.

- **AND** — both conditions must be true
- **OR** — either condition can be true
- **condition1 / condition2** — the rules you are testing

Example: customers in a city *and* with a premium plan.

Negating with NOT / !=

```
SELECT column_name
FROM table_name
WHERE column != 'value';
```

-- OR

```
SELECT column_name
FROM table_name
WHERE NOT condition;
```

Breakdown: exclude values or reverse an entire condition.

- **!=** — does not equal
- **NOT** — flips the logic of a condition
- **'value'** — the value you want to exclude

Example: show all records except students or inactive users.



Best Practices, Tips & Tricks (WHERE Clause Focus)

The Parentheses Rule

Crucial tip: `AND` is evaluated before `OR`. If you mix them in one query without parentheses, you can get the wrong results.

Wrong:

```
WHERE role = 'student' OR  
role = 'professor' AND  
is_active = true;
```

Right:

```
WHERE (role = 'student' OR  
role = 'professor') AND  
is_active = true;
```

Case Sensitivity

In PostgreSQL, text inside single quotes is case-sensitive. A small capitalization difference can return no rows at all.

Right:

```
WHERE role = 'admin'
```

Wrong:

```
WHERE role = 'Admin'
```

Nulls are Ghosts

You cannot use `=` or `!=` to check for `NULL` values. Use `IS NULL` or `IS NOT NULL` instead.

Wrong:

```
WHERE department_id =  
NULL
```

Right:

```
WHERE department_id IS  
NULL
```

Use NOT for Cleaner Filters

When you want to exclude a condition, `NOT` can often make the query easier to read and maintain.

Wrong:

```
WHERE status != 'inactive'
```

Right:

```
WHERE NOT status =  
'inactive'
```



Practice Questions - WHERE Clause

1

Active Users

Write a query to retrieve the name, email, and role of all users who are currently active in the system.

Hint: You are looking for a boolean column to be exactly true.

2

Filter by Role

Write a query to find all users whose role is 'student'.

Hint: Compare the role column directly to the text value you want.

3

Combined Conditions

Write a query to find all courses in the courses table that have a fee of greater than or equal to 2000 **AND** less than or equal to 4000.

Hint: You will need to write the column name fee twice in your WHERE clause, one for each side of the AND.

4

Target Audience

Write a query to find users who are either **inactive** (is_active = false) **OR** belong strictly to department_id = 5 (Basic Sciences).

Hint: Just string the two completely separate conditions together with the flexible OR keyword.

5

NULL Check

Write a query to find users whose department_id is NULL.

Hint: Use IS NULL instead of = NULL.

Advanced Filtering

Use **IN**, **BETWEEN**, **LIKE**, and **ILIKE** to write more precise, flexible queries and quickly narrow results from large datasets.



Filter smarter

Target exactly the rows you need.



Match patterns

Find values using partial text and case-insensitive matching.



Range and lists

Compare values across sets and intervals with confidence.

Advanced Filtering Operators



IN

The ultimate shortcut for the `OR` operator. Check whether a value matches *any* item in a comma-separated list.

- Great for matching against multiple values
- Compact alternative to long `OR` chains



BETWEEN

Filter data within a specific range. It is especially useful for numbers and dates.

- **Inclusive:** includes both start and end values
- Ideal for ranges, thresholds, and time periods



LIKE

Use pattern matching in text to find values that fit a specific format. It is **case-sensitive**.

- Matches text patterns instead of exact values
- Supports wildcards: `%` for zero or more characters, `_` for exactly one character



ILIKE

PostgreSQL's case-insensitive version of `LIKE`. Perfect for user search bars and flexible text matching.

- Works like `LIKE`, but ignores letter case
- Also supports wildcards: `%` and `_`

SQL Filtering Patterns

IN — Searching Lists

```
SELECT columns
FROM table_name
WHERE column IN (value1,
value2, value3);
```

Use **IN** when you want to match a value against one of several specific options.

BETWEEN — Searching Ranges

```
SELECT columns
FROM table_name
WHERE column BETWEEN
start_value AND end_value;
```

Use **BETWEEN** when filtering values within an inclusive range, such as dates or numbers.

LIKE / ILIKE — Pattern Matching

```
SELECT columns
FROM table_name
WHERE string_column ILIKE
'%pattern%';
```

Use **LIKE** for case-sensitive matching and **ILIKE** for case-insensitive text search.

Best Practices, Tips & Tricks

1. IN vs = gotcha ⚠️

You cannot use `=` with a list. If you have parentheses with multiple values, you **must** use `IN`.

```
WHERE id IN (1, 2, 3)
```

Wrong: `WHERE id = (1, 2, 3)` will throw a syntax error.

2. Leading wildcard performance killer 🚨

Writing `ILIKE '%sharma'` forces the database to scan every row one by one, bypassing indexes. Think of it as a **slow query** warning for massive databases.

```
WHERE name ILIKE  
'%sharma'
```

We will learn how to fix this in Day 22.

3. Underscore wildcard usage 🔍

Don't forget `_`. It matches **exactly one character**, so it is useful when you want a specific name length or pattern.

```
WHERE name ILIKE 'A__'
```

This finds names that are exactly 4 letters long and start with A.

Practice Questions

Advanced Filtering exercises to test IN, BETWEEN, and case-insensitive matching.

1

The ID Lottery

Select the `id`, `name`, and `email` of users whose IDs are exactly 1, 4, 7, 15, and 22 using a single operator.

Sweet Hint: Don't write five different OR statements. Wrap those numbers in parentheses and use the keyword that means "inside this list".

2

The Registration Window

Find all users whose `created_at` timestamp falls between '2026-04-01' and '2026-04-30'.

Sweet Hint: Use the keyword that sits "in the middle" of two values, and remember to connect your start and end dates with an AND.

3

The A-List

Find all users whose names start with A or a, regardless of capitalization.

Sweet Hint: You need PostgreSQL's special case-insensitive pattern matcher. Follow it with the letter 'A' and a percentage sign to signify "starts with A, and anything can come after."

Sorting and Paginating

1

ORDER BY

Sorts your results based on one or more columns.

Example: ORDER BY created_at

2

ASC

Sorts from lowest to highest. This is the default if you do not specify a direction.

Example: ORDER BY name ASC

3

DESC

Sorts from highest to lowest.

Example: ORDER BY created_at DESC

4

LIMIT

Restricts the total number of rows returned.

Example: LIMIT 10

5

OFFSET

Skips a specific number of rows before returning results. This is key for pagination.

Example: OFFSET 20

SQL Patterns for Result Control

Use these patterns to sort, trim, and page through query results with clarity.

A

Sorting Data



```
SELECT columns
FROM table_name
ORDER BY
column_name ASC; -- or
DESC
```

Sorts query results by one or more columns in ascending or descending order.

B

Limiting Results



```
SELECT columns
FROM table_name
LIMIT number_of_rows;
```

Restricts how many rows the database returns, which helps keep queries efficient.

C

Pagination Formula



```
SELECT columns
FROM table_name
ORDER BY column_name
LIMIT amount_per_page
OFFSET rows_to_skip;
```

Combines sorting, limit, and offset to fetch one page of results at a time.

Best Practices, Tips & Tricks (Day 9 Focus)

Three practical patterns to keep your PostgreSQL queries predictable, readable, and scalable.

🚫 Never use `LIMIT` without `ORDER BY`

If you just write `SELECT * FROM users LIMIT 5;`, PostgreSQL can return rows in an unpredictable order based on how they are stored on disk. Always anchor `LIMIT` with a specific `ORDER BY` so results are consistent every time.

```
SELECT *  
FROM users  
ORDER BY created_at DESC  
LIMIT 5;
```

🧩 The `NULLS LAST` trick

In PostgreSQL, `NULL` values are treated like the "largest" possible value. If you sort `ASC`, rows with `NULL` drop to the bottom. If you sort `DESC`, `NULL`s appear at the top. You can fix that with:

```
ORDER BY department_id DESC  
NULLS LAST;
```

⚠️ The `OFFSET` performance warning

`OFFSET` is fine for small tables. But if you have 10 million rows and write `OFFSET 9000000`, the database still has to scan and skip all 9 million rows, which is slow. For massive apps, use cursor-based pagination instead:

```
WHERE id > last_seen_id
```

Practice Questions

Sorting & Paginating

1

1. The Roll Call

Write a query to retrieve all users from the `users` table, sorted strictly alphabetically by their `name` from A to Z.

Sweet Hint: Sorting from A to Z is the default behavior, but try to use the explicit keyword `ASC` just to build good habits!

2

2. The Premium Catalog

Write a query to fetch the top 5 most expensive courses from the `courses` table, showing only their `title` and `fee`.

Sweet Hint: You need to sort the `fee` backwards (highest to lowest) and then use a keyword to chop off the rest of the list after the 5th row.

3

3. Dashboard Page 2

You are building a datatable that shows 10 courses per page, ordered by their `id`. Write the query to fetch exactly Page 2 of this list (items 11 through 20).

Sweet Hint: You need to `LIMIT` the results to 10, but you also need to use the magic keyword that tells PostgreSQL to "skip" the first 10 records.